

Laboratório de Traceroute com ICMP

Neste laboratório, você aprenderá como implementar uma aplicação de traceroute usando mensagens de solicitação e resposta ICMP. Recomenda-se fortemente que você realize primeiro o laboratório de ICMP Pinger antes deste, pois ambos seguem a mesma abordagem. O cálculo do *checksum* e a criação do cabeçalho não são abordados neste laboratório, sendo necessário consultar o laboratório de ping para esses detalhes. A nomeação de variáveis e sockets também é similar.

O **Traceroute** é uma ferramenta de diagnóstico de rede que permite a um usuário rastrear o caminho entre um host que executa o programa e qualquer outro host no mundo. Ele é implementado com mensagens ICMP. O funcionamento consiste em enviar mensagens de eco ICMP (tipo 8) para o mesmo destino, aumentando progressivamente o valor do campo TTL (Time-to-Live). Os roteadores ao longo do caminho retornam mensagens ICMP Time Exceeded (tipo 11) quando o TTL atinge zero. O destino final envia uma resposta ICMP (tipo 0) ao receber a solicitação de eco. Os endereços IP dos roteadores que enviam respostas podem ser extraídos dos pacotes recebidos. O tempo de ida e volta (round-trip time) entre o host de origem e o roteador é calculado com um cronômetro no host de origem.

Sua tarefa é desenvolver sua própria aplicação Traceroute em Python usando ICMP. Para simplificar, sua aplicação não seguirá exatamente a especificação oficial do RFC 1739.

Código

Abaixo está o código esqueleto para o cliente. Você deve completá-lo. Os pontos que precisam ser preenchidos estão marcados com `#Fill in start` e `#Fill in end`. Cada ponto pode exigir uma ou mais linhas de código.

Notas adicionais:

1. Não é necessário se preocupar com o checksum, pois ele já é fornecido no código.
2. Este laboratório requer o uso de **sockets raw**. Em alguns sistemas operacionais, pode ser necessário ter privilégios de administrador/root para executar o programa.
3. Consulte o exercício de programação **ICMP Pinger** no final do Lab 4 para mais informações sobre ICMP.
4. Este programa pode não funcionar para sites que bloqueiam tráfego ICMP.
5. Pode ser necessário desativar o firewall ou o software antivírus para permitir o envio e recebimento das mensagens.

Apresentação

Em sua apresentação, você deve mostrar e explicar o código completo do cliente e mostrar o ping para quatro hosts de destino, cada um em um continente diferente.

Esta atividade vale nota. A implementação básica resulta em nota máxima 6,0.

Código Esqueleto em Python para ICMP Traceroute:

```
from socket import *
import os
import sys
import struct
import time
```

```

import select
import binascii

ICMP_ECHO_REQUEST = 8
MAX_HOPS = 30
TIMEOUT = 2.0
TRIES = 2

def checksum(string):
    # Função para calcular o checksum do pacote
    # Dica: veja o laboratório de ICMP Ping

def build_packet():
    # Criar o cabeçalho como no exercício de ping
    # Adicionar o checksum ao cabeçalho
    # Retornar o pacote completo
    packet = header + data
    return packet

def get_route(hostname):
    timeLeft = TIMEOUT
    for ttl in range(1, MAX_HOPS):
        for tries in range(TRIES):
            destAddr = gethostbyname(hostname)
            # #Fill in start - Criar um socket raw chamado mySocket
            # #Fill in end
            mySocket.setsockopt(IPPROTO_IP, IP_TTL, struct.pack('I', ttl))
            mySocket.settimeout(TIMEOUT)

            try:
                d = build_packet()
                mySocket.sendto(d, (hostname, 0))
                t = time.time()
                startedSelect = time.time()
                whatReady = select.select([mySocket], [], [], timeLeft)
                howLongInSelect = (time.time() - startedSelect)

                if whatReady[0] == []:
                    print(" * * * Request timed out.")

                recvPacket, addr = mySocket.recvfrom(1024)
                timeReceived = time.time()
                timeLeft -= howLongInSelect

                if timeLeft <= 0:
                    print(" * * * Request timed out.")

            except timeout:
                continue

            else:
                # #Fill in start - Obter o tipo ICMP do pacote IP
                # #Fill in end

                if types == 11:
                    bytes = struct.calcsize("d")
                    timeSent = struct.unpack("d", recvPacket[28:28 + bytes])[0]
                    print(" %d rtt=%.0f ms %s" % (ttl, (timeReceived - t) *
1000, addr[0]))

                elif types == 3:
                    bytes = struct.calcsize("d")
                    timeSent = struct.unpack("d", recvPacket[28:28 + bytes])[0]
                    print(" %d rtt=%.0f ms %s" % (ttl, (timeReceived - t) *
1000, addr[0]))

                elif types == 0:
                    bytes = struct.calcsize("d")
                    timeSent = struct.unpack("d", recvPacket[28:28 + bytes])[0]
                    print(" %d rtt=%.0f ms %s" % (ttl, (timeReceived - timeSent)
* 1000, addr[0]))

                return

            else:
                print("error")

            break

        finally:
            mySocket.close()

get_route("google.com")

```

Exercícios Opcionais:

Atualmente, a aplicação imprime apenas uma lista de endereços IP de todos os roteadores ao longo do caminho. Tente usar o método `gethostbyname` para imprimir os nomes de cada rota intermediária.